

Lab 2: for a given set of training data examples stored in a .csv file, implement and demonstrate Candidate Elimination algorithm to output a description of the set of all hypothesis consistent with training examples

Candidate Elimination Algorithm

The Candidate Elimination algorithm finds the **version space** — the set of all hypotheses that are consistent with the given training examples.

It maintains two boundaries:

- **S (Specific boundary)**: Most specific hypotheses consistent with the training data.
- **G (General boundary)**: Most general hypotheses consistent with the training data.

For a typical dataset such as:

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Hypothesis representation

<Sky, AirTemp, Humidity, Wind, Water, Forecast>

A ? means:

Any value is acceptable.

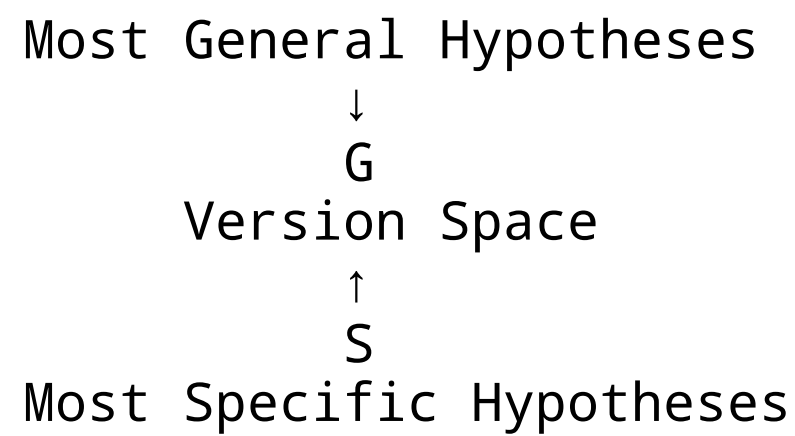
For example:

<Sunny, Warm, ?, Strong, ?, ?>

means:

- Sky must be Sunny
- Air temperature must be Warm
- Humidity can be anything
- Wind must be Strong
- Water can be anything
- Forecast can be anything

The algorithm therefore searches between:



Candidate Elimination does **not** simply produce one hypothesis. It produces the **set of all hypotheses consistent with the training examples**, represented by the S and G boundaries.

Program Code:

```
import pandas as pd
```

```
# Read training data from CSV file
```

```
data = pd.read_csv("EnjoySport.csv")
```

```
print("Training Data:")
```

```
print(data)
```

```
# Separate attributes and target
```

```
attributes = data.columns[:-1]
```

```
target = data.columns[-1]
```

```
# Initialize Specific boundary
```

```
S = ['0'] * len(attributes)
```

```
# Initialize General boundary
```

```
G = [['?'] * len(attributes)]
```

```
# Check whether hypothesis covers example
```

```
def covers(hypothesis, example):
```

```
    for h, e in zip(hypothesis, example):
```

```
        if h != '?' and h != e:
```

```
            return False
```

```
    return True
```

```
# Generalize S to cover positive example
```

```
def generalize_S(S, example):
```

```
    new_S = S.copy()
```

```
    for i in range(len(S)):
```

```
        if S[i] == '0':
```

```
            new_S[i] = example[i]
```

```
        elif S[i] != example[i]:
```

```
            new_S[i] = '?'
```

```
    return new_S
```



```
# Specialize G to exclude negative example
```

```
def specialize_G(G, S, example):
```

```
    new_G = []
```

```
    for g in G:
```

```
        for i in range(len(attributes)):
```

```
            if g[i] == '?':
```

```
                if S[i] != '?':
```

```
                    new_h = g.copy()
```

```
                    new_h[i] = S[i]
```

```
                    # Keep only hypotheses that do not cover negative example
```

```
                    if not covers(new_h, example):
```

```
                        new_G.append(new_h)
```

```
    return new_G
```

```
# Candidate Elimination Algorithm
```

```
for index, row in data.iterrows():
```

```
    example = row.iloc[:-1].tolist()
```

```
    label = row.iloc[-1]
```

```
    print("\nExample:", example)
```

```
print("Target:", label)
```

```
if label == "Yes":
```

```
    # Remove hypotheses from G that do not cover positive example
```

```
    G = [g for g in G if covers(g, example)]
```

```
    # Generalize S
```

```
    S = generalize_S(S, example)
```

```
else:
```

```
    # Remove hypotheses from G that cover negative example
```

```
    G = [g for g in G if not covers(g, example)]
```

```
    # Specialize G
```

```
    G = specialize_G(G, S, example)
```

```
print("S =", S)
```

```
print("G =", G)
```

```
print("\nFinal Version Space Boundaries")
```

```
print("-----")
```

```
print("Specific Boundary (S):")
```

```
print(S)
```

```
print("\nGeneral Boundary (G):")
```

```
for g in G:
```

```
    print(g)
```